

SÉCURISATION DU PROTOCOLE FTP

Mise en œuvre d'une infrastructure FTPS avec PKI

Enseignant : Youssouf Moussa ADOUM
Réalisé par le binôme : SENE Madjiguene & MEDOR Alexandre
Date de rendu : 18 Décembre 2025

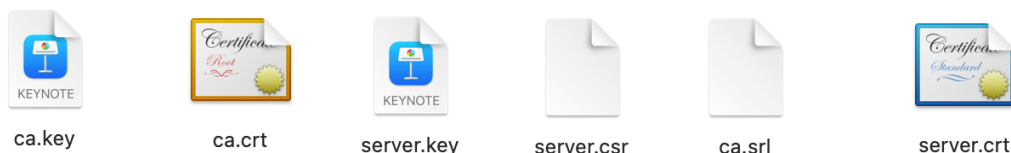


FIGURE 1 – Les fichiers générés

*Rapport technique détaillant les concepts théoriques, l'implémentation de la PKI,
le durcissement du serveur et l'analyse de trafic.*

Table des matières

1	Introduction	2
2	Infrastructure PKI (Public Key Infrastructure)	2
2.1	Création de l'Autorité de Certification (CA)	2
2.2	Génération et Signature du Certificat Serveur	3
3	Déploiement Sécurisé des Clés	4
4	Configuration du Serveur (Kali Linux)	5
5	Analyse de Trafic et Validation	6
5.1	Le problème : FTP en clair	6
5.2	La solution : Configuration et Mise en place du FTPS	8
6	Conclusion Générale	10

Table des figures

1	Les fichiers générés	1
2	Génération de la clé CA et du certificat racine	3
3	Génération de la clé serveur, du CSR et signature par la CA	4
4	Installation des clés, sécurisation (chmod 600) et création utilisateur	6
5	Connexion FTP standard (non sécurisée) réussie	7
6	Capture Wireshark : Mot de passe visible en clair (PASS)	7
7	Configuration du fichier /etc/vsftpd.conf avec SSL activé	8
8	Logs FileZilla confirmant l'établissement de la connexion TLS	9
9	Trafic chiffré dans Wireshark (TLS v1.3)	9

1 Introduction

Dans un contexte numérique où les échanges de données sont omniprésents, la sécurité des transferts de fichiers est devenue un enjeu critique pour les organisations.

Le protocole FTP (*File Transfer Protocol*), bien que standard et largement utilisé pour l'échange de documents, souffre d'une vulnérabilité majeure par conception : l'absence de chiffrement.

Dans sa configuration par défaut, toutes les informations, y compris les identifiants de connexion et le contenu des fichiers, circulent "en clair" sur le réseau, exposant le système à des attaques de type "Man-in-the-Middle" (interception).

L'objectif de ce projet est de sécuriser cette infrastructure en déployant le protocole FTPS (*FTP over SSL/TLS*).

Cette démarche ne se limite pas à une simple installation logicielle, mais nécessite la mise en place d'une chaîne de confiance cryptographique complète.

💡 Concept Théorique : Réalisations

Pour mener à bien cette mission, nous avons structuré notre approche en trois phases principales :

1. **La mise en place d'une Infrastructure à Clés Publiques (PKI) :** Nous avons agi en tant que notre propre Autorité de Certification (Mini-CA) pour générer et signer les certificats numériques nécessaires via la boîte à outils **OpenSSL**.
2. **L'audit de vulnérabilité et la configuration serveur :** Nous avons d'abord mis en évidence les failles du FTP standard via une analyse de trafic réseau avec **Wireshark**, avant de configurer le serveur **vsftpd** sous Kali Linux pour imposer le chiffrement.
3. **La validation de la sécurité :** Nous avons vérifié l'efficacité du chiffrement SSL/TLS en analysant à nouveau le trafic réseau, garantissant ainsi la confidentialité et l'intégrité des échanges.

Ce rapport détaille les étapes techniques, les commandes réalisées et les résultats obtenus lors de cette sécurisation.

2 Infrastructure PKI (Public Key Infrastructure)

Pour sécuriser les échanges, nous ne pouvons pas nous contenter de certificats auto-signés simples qui déclenchent des alertes de sécurité. Nous avons simulé une architecture réelle en créant notre propre Autorité de Certification.

💡 Concept Théorique : Pourquoi RSA et AES ?

Dans cette étape, nous utilisons deux familles d'algorithmes cryptographiques :

- **RSA (Rivest-Shamir-Adleman) 4096 bits :** Un algorithme asymétrique utilisé pour l'identité de la CA. Une taille de 4096 bits est choisie pour garantir une sécurité à long terme (supérieure aux 2048 bits standards).
- **AES-256 (Advanced Encryption Standard) :** Utilisé ici pour chiffrer la clé privée de la CA sur le disque ("at rest"). Sans le mot de passe, la clé privée est inutilisable même si elle est volée.

2.1 Création de l'Autorité de Certification (CA)

Nous générons d'abord la clé privée de l'autorité, protégée par mot de passe, puis son certificat racine.

```

(base) madjiguenedotesene@MacBook-Air-de-Madjiguene projet_secu % openssl genrsa -aes256 -out ca.key 4096
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
(base) madjiguenedotesene@MacBook-Air-de-Madjiguene projet_secu % openssl req -new -x509 -sha256 -days 3650 -key ca.key -out ca.crt
Enter pass phrase for ca.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:Fr
State or Province Name (full name) [Some-State]:Ile-de-France
Locality Name (eg, city) []:Saint-Denis
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Université Paris8 Projet Securite
Organizational Unit Name (eg, section) []:Mini-CA Racine
Common Name (e.g. server FQDN or YOUR name) []:Mini-CA du Projet FTPS
Email Address []:madjiguenesene20@gmail.com
-----

```

FIGURE 2 – Génération de la clé CA et du certificat racine

Ce terminal illustre la création des fondations d'une Autorité de Certification (CA) racine. Nous avons généré l'identité numérique qui servira à valider et signer d'autres certificats de sécurité.

💡 Concept Théorique : Mini-CA capable d'émettre des certificats

- **Création de la clé privée (ca.key) :**
 - La commande `genrsa` a permis de générer une clé RSA très robuste (4096 bits).
 - L'option `-aes256` a chiffré cette clé avec une "PEM pass phrase" pour en sécuriser l'accès.
- **Création du certificat racine (ca.crt) :**
 - La commande `req -x509` a utilisé notre clé privée pour générer un certificat auto-signé.
 - Nous avons défini sa durée de validité à 10 ans (`-days 3650`).
 - Nous avons renseigné les informations d'identité de l'autorité (Pays : FR, Organisation : Université Paris8, Nom commun : Mini-CA du Projet FTPS).

2.2 Génération et Signature du Certificat Serveur

Contrairement à la CA, la clé du serveur FTP ne doit pas être protégée par mot de passe pour permettre le redémarrage automatique des services (systemd).

💡 Concept Théorique : Le format X.509 et le CSR

Le processus respecte le standard X.509 :

1. **Clé Privée** : Identité secrète du serveur.
2. **CSR (Certificate Signing Request)** : Contient les informations publiques (Pays, Organisation, CN=IP du serveur).
3. **Certificat (CRT)** : Le CSR est signé par la CA (avec `-CAkey`), créant une chaîne de confiance valide.

```

(base) madjiguenedotesene@MacBook-Air-de-Madjiguene projet_secu % openssl genrsa -out server.key 2048
(base) madjiguenedotesene@MacBook-Air-de-Madjiguene projet_secu % openssl req -new -key server.key -out server.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:Fr
State or Province Name (full name) [Some-State]:Ile-de-France
Locality Name (eg, city) []:Saint-Denis
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Université Paris8 Projet Securite
Organizational Unit Name (eg, section) []:Mini-CA Racine
Common Name (e.g. server FQDN or YOUR name) []:Mini-CA du Projet FTPS
Email Address []:madjiguenesene20@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:dote2000
An optional company name []:
(base) madjiguenedotesene@MacBook-Air-de-Madjiguene projet_secu % openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
server.crt -days 365 -sha256
Certificate request self-signature ok
subject=C=Fr, ST=Ile-de-France, L=Saint-Denis, O=Universit  Paris8 Projet Securite, OU=Mini-CA Racine, CN=Mini-CA du Projet FTPS, emailAddress=
madjiguenesene20@gmail.com
Enter pass phrase for ca.key:
(base) madjiguenedotesene@MacBook-Air-de-Madjiguene projet_secu %

```

FIGURE 3 – Génération de la clé serveur, du CSR et signature par la CA

Dans cette étape, nous avons procédé à la création de l'identité du serveur FTPS et à sa validation par notre Autorité de Certification. Voici le détail des opérations effectuées :

💡 Concept Théorique : Création de l'identité du serveur FTPS

- **Génération de la clé privée du serveur (server.key) :**
 - Nous avons généré une clé RSA de 2048 bits.
 - Contrairement à la clé du CA, celle-ci n'est pas chiffrée par un mot de passe (pas d'option `-aes256`), ce qui permet au serveur de démarrer automatiquement sans intervention humaine.
- **Création de la demande de signature (server.csr) :**
 - Nous avons créé un fichier CSR (*Certificate Signing Request*) contenant les informations du serveur.
 - Point crucial : pour le "Common Name", nous avons spécifié l'adresse IP du serveur (**172.20.10.50**) afin que le certificat soit valide pour cette machine spécifique.
- **Signature et émission du certificat (server.crt) :**
 - Nous avons utilisé notre propre CA (fichiers `ca.crt` et `ca.key`) pour signer la demande du serveur.
 - Nous avons défini une validité de 1 an (`-days 365`).
 - L'option `-CAcreateserial` a permis de générer un numéro de série unique pour ce certificat.

Résultat : Nous disposons désormais d'un certificat serveur (`server.crt`) signé et validé par notre propre autorité, prêt à être installé sur le serveur FTPS.

3 Déploiement Sécurisé des Clés

Le transfert des clés vers le serveur doit être sécurisé. Nous utilisons `scp` (Secure Copy Protocol) basé sur SSH.

Session Terminal

```
(base) madjiguenedotesene@MacBook-Air-de-Madjiguene Protocole_de_securite % echo "
  Transfert de server.key..."
scp server.key kali@172.20.10.50:/home/kali/
The authenticity of host '172.20.10.50' can't be established.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
kali@172.20.10.50's password:
server.key                                100% 1704      1.6MB/s   00:00

(base) madjiguenedotesene@MacBook-Air-de-Madjiguene Protocole_de_securite % echo "
  Transfert de server.crt..."
scp server.crt kali@172.20.10.50:/home/kali/
server.crt                                100% 1923      1.2MB/s   00:00

(base) madjiguenedotesene@MacBook-Air-de-Madjiguene Protocole_de_securite % scp ca.
  crt kali@172.20.10.50:/home/kali/
ca.crt                                    100% 2325      1.3MB/s   00:00
```

Cette étape illustre le déploiement des fichiers cryptographiques, générés localement, vers le serveur cible qui hébergera le service FTPS. Voici les actions que nous avons effectuées :

💡 Concept Théorique : Transfère des Fichiers

— Utilisation du protocole SCP :

- Nous avons employé la commande `scp` (Secure Copy) pour transférer les fichiers de manière sécurisée via SSH de notre machine locale (MacBook) vers le répertoire `/home/kali/` de la machine distante (IP : **172.20.10.50**).

— Établissement de la connexion :

- Lors de la première tentative de connexion, nous avons validé l'empreinte (fingerprint) de la machine distante pour confirmer son identité SSH.
- Nous nous sommes ensuite authentifiés en saisissant le mot de passe de l'utilisateur `kali`.

— Transfert des composants du certificat :

- Nous avons transféré les trois éléments indispensables : la clé privée du serveur (`server.key`), le certificat public du serveur (`server.crt`) et le certificat de notre autorité racine (`ca.crt`).

Résultat : Tous les fichiers cryptographiques nécessaires sont désormais présents sur la machine Kali Linux, prêts à être intégrés dans la configuration du serveur FTP.

4 Configuration du Serveur (Kali Linux)

Sur la machine cible (Kali), nous appliquons le principe de moindre privilège.

```

(kali㉿kali)-[~]
$ sudo mv /home/kali/ca.crt /etc/ssl/vsftpd/
[sudo] password for kali:

(kali㉿kali)-[~]
$ ls -l /etc/ssl/vsftpd/
total 12
-rw-r--r-- 1 kali kali 2325 Nov 18 09:11 ca.crt
-rw-r--r-- 1 kali kali 1923 Nov 18 08:51 server.crt
-rw----- 1 kali kali 1704 Nov 18 08:50 server.key

(kali㉿kali)-[~]
$ sudo chmod 600 /etc/ssl/vsftpd/server.key

(kali㉿kali)-[~]
$ sudo adduser majiguene
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for majiguene
Enter the new value, or press ENTER for the default
  Full Name []: majiguene
  Room Number []: 162
  Work Phone []: 0605731763
  Home Phone []:
  Other []:
Is the information correct? [Y/n] Y

(kali㉿kali)-[~]
$

```

FIGURE 4 – Installation des clés, sécurisation (chmod 600) et création utilisateur

Après avoir transféré les fichiers sur la machine Kali Linux, nous avons procédé à la mise en place de l'environnement sécurisé pour le serveur FTP. Voici les actions réalisées dans ce terminal :

💡 Concept Théorique : Permissions chmod 600

- **Organisation des fichiers cryptographiques :**
 - Nous avons créé un répertoire dédié et structuré : `/etc/ssl/vsftpd/`.
 - Nous y avons déplacé les trois fichiers essentiels (`server.key`, `server.crt`, et `ca.crt`) afin de les regrouper dans un emplacement système standard, séparé du dossier personnel de l'utilisateur.
- **Durcissement des permissions (Sécurité Critique) :**
 - Nous avons appliqué la commande `chmod 600` sur la clé privée (`server.key`).
 - Cette étape est fondamentale : elle garantit que seul l'administrateur (root) peut lire ce fichier. Si cette clé était accessible aux autres utilisateurs, la sécurité du chiffrement serait compromise.
- **Création de l'utilisateur FTP :**
 - Nous avons créé un utilisateur spécifique nommé `majiguene` via la commande `adduser`.
 - Cet utilisateur servira à tester la connexion authentifiée une fois le serveur configuré.

Résultat : L'environnement serveur est prêt et sécurisé. Les certificats sont à leur place définitive et l'utilisateur de test est opérationnel. Nous pouvons désormais passer à la configuration du service vsftpd.

5 Analyse de Trafic et Validation

5.1 Le problème : FTP en clair

Avant d'implémenter la couche de sécurité SSL/TLS, nous avons réalisé un audit de vulnérabilité sur la configuration par défaut du protocole FTP. Cette démonstration met en évidence les risques critiques d'une transmission non chiffrée.

```

(kali@kali)-[~]
$ sudo systemctl restart vsftpd
[sudo] password for kali:

(kali@kali)-[~]
$ ftp 172.20.10.50
Connected to 172.20.10.50.
220 (vsFTPd 3.0.5)
Name (172.20.10.50:kali): majiguene
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||51819|)
150 Here comes the directory listing.
226 Directory send OK.
ftp> exit
221 Goodbye.

```

FIGURE 5 – Connexion FTP standard (non sécurisée) réussie

⚠ Vulnérabilité Identifiée : Interception de mot de passe (Sniffing)

Avant sécurisation, le protocole FTP transmet les commandes en texte clair. Une capture Wireshark montre le mot de passe sans aucun effort de déchiffrement.

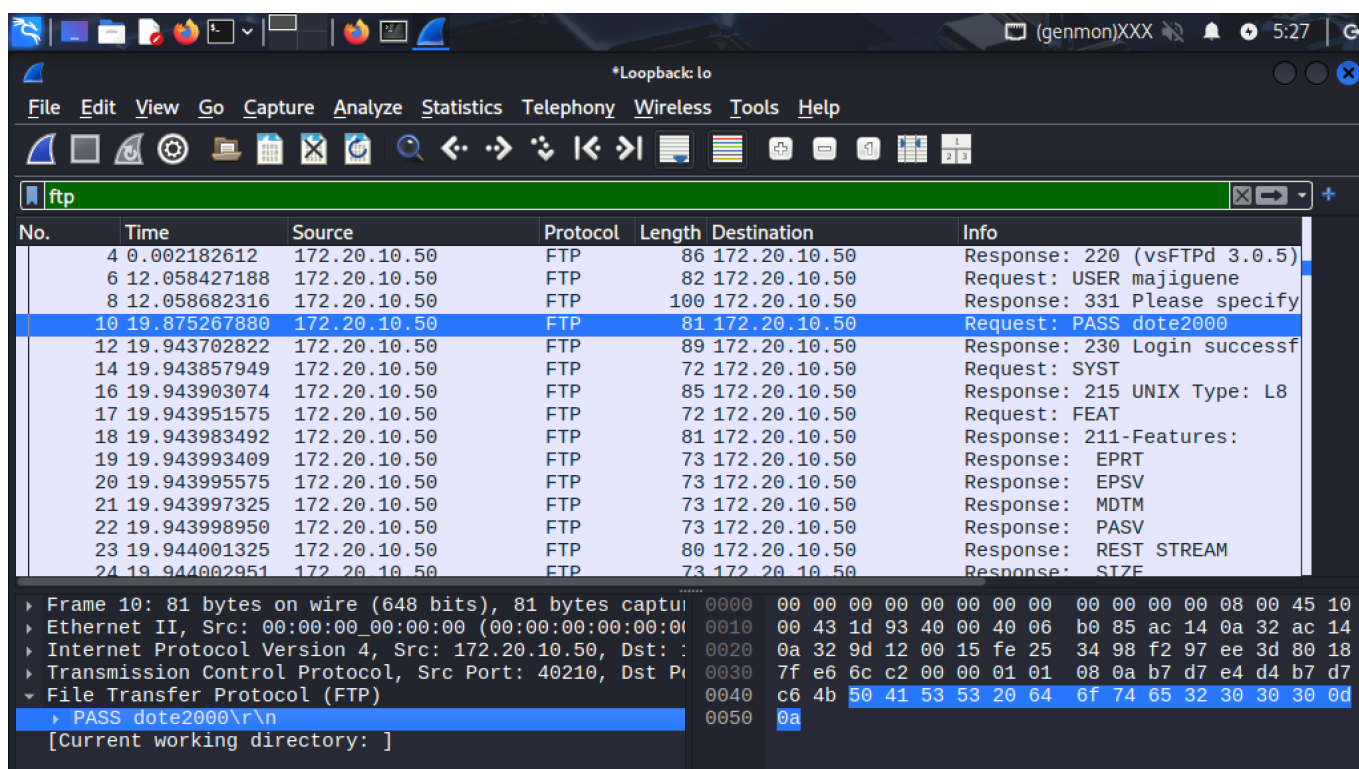


FIGURE 6 – Capture Wireshark : Mot de passe visible en clair (PASS)

💡 Concept Théorique : Analyse

— Test fonctionnel de connexion :

- Nous nous sommes connectés au serveur (172.20.10.50) avec l'utilisateur `majiguene` créé précédemment.
- La connexion s'établit correctement et le transfert de fichier fonctionne, confirmant que le service est opérationnel.

— Interception de trafic (Sniffing) :

- Parallèlement à la connexion, nous avons lancé une capture de paquets réseaux avec l'outil **Wireshark**.

- L'analyse des trames révèle une faille de sécurité majeure inhérente au protocole FTP standard : l'absence totale de chiffrement.

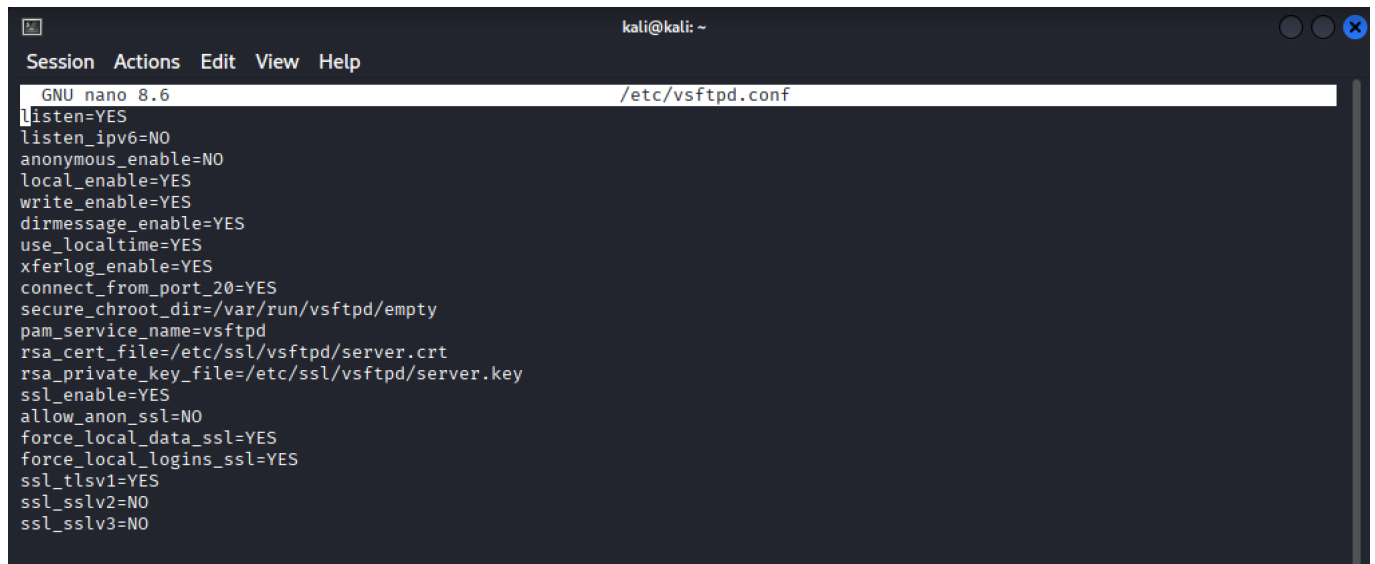
⚠ Vulnérabilité Identifiée : Constat critique :

- Comme le montre la capture, les identifiants d'authentification circulent "en clair" sur le réseau. Nous pouvons lire distinctement la commande USER madjiguene et surtout PASS dote2000.
- De plus, le contenu même des fichiers transférés est également lisible par n'importe quel attaquant écoutant le réseau (Man-in-the-Middle).

Conclusion : Cette étape valide la nécessité impérative de passer au protocole FTPS pour garantir la confidentialité des données et de l'authentification.

5.2 La solution : Configuration et Mise en place du FTPS

Nous passons maintenant à la sécurisation effective du service en activant le module SSL/TLS dans le fichier de configuration principal.



```
Session Actions Edit View Help
GNU nano 8.6 /etc/vsftpd.conf
listen=YES
listen_ipv6=NO
anonymous_enable=NO
local_enable=YES
write_enable=YES
dirmessage_enable=YES
use_localtime=YES
xferlog_enable=YES
connect_from_port_20=YES
secure_chroot_dir=/var/run/vsftpd/empty
pam_service_name=vsftpd
rsa_cert_file=/etc/ssl/vsftpd/server.crt
rsa_private_key_file=/etc/ssl/vsftpd/server.key
ssl_enable=YES
allow_anon_ssl=NO
force_local_data_ssl=YES
force_local_logins_ssl=YES
ssl_tlsv1=YES
ssl_sslv2=NO
ssl_sslv3=NO
```

FIGURE 7 – Configuration du fichier /etc/vsftpd.conf avec SSL activé

Après activation du chiffrement, nous utilisons le client graphique FileZilla car le client FTP standard en ligne de commande ne gère pas toujours le chiffrement implicite facilement. Lors de la connexion, nous observons le Handshake TLS (négociation de sécurité).

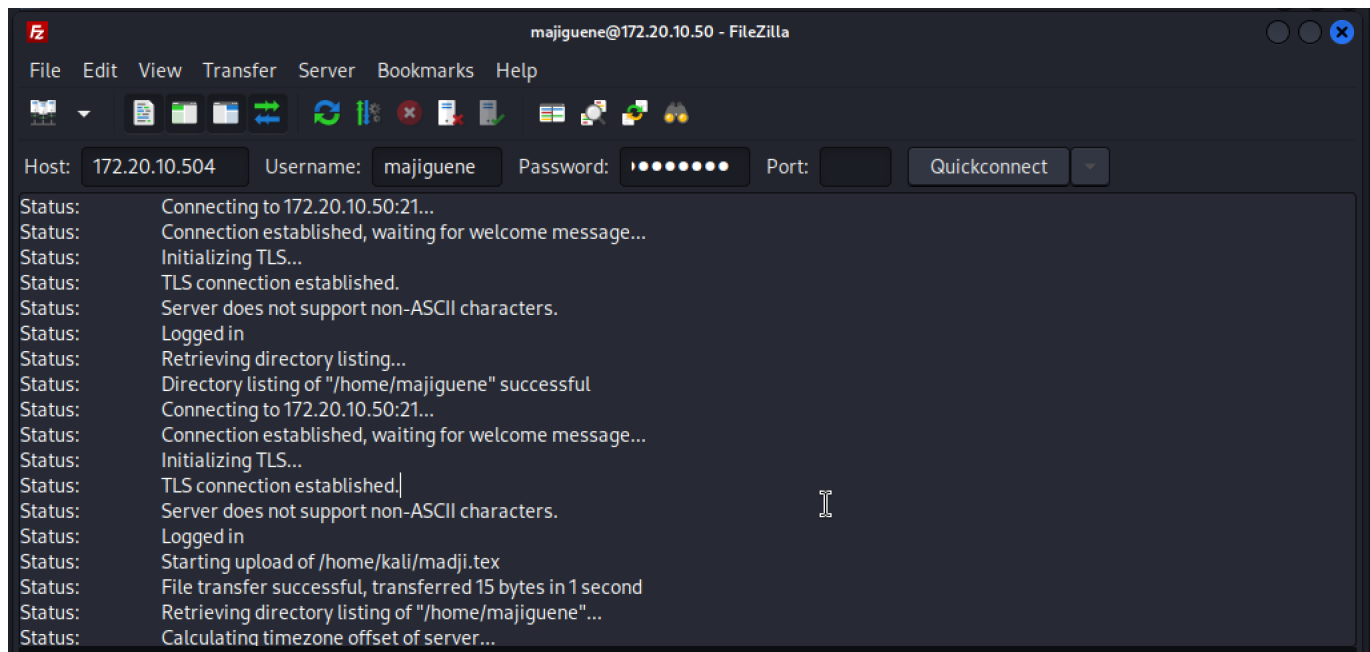


FIGURE 8 – Logs FileZilla confirmant l'établissement de la connexion TLS

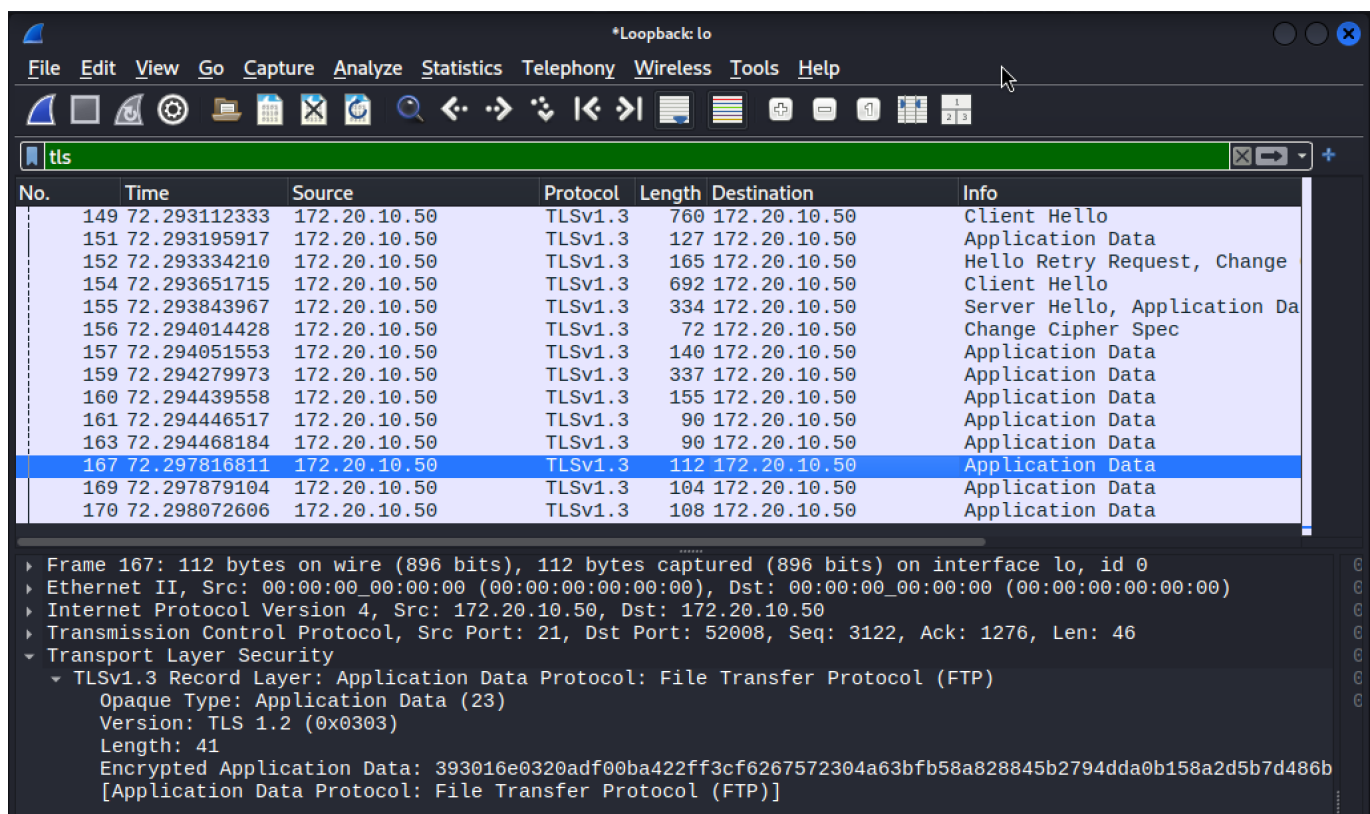


FIGURE 9 – Trafic chiffré dans Wireshark (TLS v1.3)

Enfin, pour valider l'efficacité de la mesure, nous avons analysé à nouveau le trafic réseau. Contrairement au test précédent, les données sont désormais illisibles.

Cette phase finale concrétise la sécurisation de notre architecture de transfert de fichiers. Nous avons transformé un service FTP vulnérable en un service FTPS robuste. Voici le détail de notre intervention :

💡 Concept Théorique : Chiffrement TLS Actif

— Configuration du moteur SSL (vsftpd.conf) :

- Nous avons indiqué au serveur l'emplacement des certificats cryptographiques générés précé-

demment (`rsa_cert_file` et `rsa_private_key_file`).

- Nous avons activé le chiffrement (`ssl_enable=YES`) et, point crucial, nous avons **forcé** l'utilisation du SSL pour l'authentification et le transfert de données (`force_local_logins_ssl=YES`, `force_local_data_ssl=YES`). Cela empêche toute connexion non sécurisée accidentelle.
- Nous avons également désactivé les protocoles obsolètes (SSLv2/SSLv3) au profit du TLS v1.2+, plus sécurisé.
- **Validation de la connexion (Client FileZilla) :**
 - Lors de la connexion, le client FileZilla a détecté le certificat. Comme il s'agit d'un certificat auto-signé par notre "Mini-CA", une alerte de sécurité est apparue (comportement normal), nous demandant de valider manuellement l'empreinte du certificat.
 - Les logs montrent clairement la commande `AUTH TLS` suivie du succès de l'initialisation TLS.

Sécurité Validée : Preuve d'imperméabilité (Wireshark) :

- La nouvelle capture de paquets confirme le succès de l'opération. Là où nous voyions précédemment les mots de passe en clair, nous ne voyons plus que des paquets "Application Data" chiffrés via le protocole TLS v1.2/1.3.
- Il est désormais mathématiquement impossible pour un attaquant situé sur le réseau de lire les identifiants ou le contenu des fichiers transférés.

6 Conclusion Générale

Ce projet nous a permis de mettre en œuvre, de bout en bout, une solution de transfert de fichiers sécurisée au sein d'un environnement Linux. En partant d'un constat de vulnérabilité critique sur le protocole FTP standard, nous avons déployé une architecture FTPS robuste basée sur le chiffrement SSL/TLS.

La démarche adoptée, allant de la création d'une Autorité de Certification (CA) racine jusqu'à la configuration fine du service `vsftpd`, a mis en lumière l'importance d'une chaîne de confiance maîtrisée pour garantir la confidentialité et l'intégrité des données en entreprise.

Concept Théorique : Bilan des Acquis

La réalisation de ce projet nous a permis d'acquérir et de consolider plusieurs compétences techniques clés :

- **Cryptographie appliquée (PKI) :** Compréhension du fonctionnement des clés asymétriques (RSA), création de demandes de signature (CSR) et gestion d'une Autorité de Certification avec **OpenSSL**.
- **Administration Système Linux :** Gestion des services (`systemctl`), manipulation sécurisée des fichiers (SCP) et gestion stricte des permissions (`chmod 600`) pour protéger les clés privées.
- **Analyse Réseau et Audit :** Utilisation de **Wireshark** pour inspecter le trafic réseau. Nous avons appris à identifier des données en clair (mots de passe) et à vérifier l'efficacité du chiffrement (TLS Handshake).
- **Configuration de Services :** Paramétrage avancé du serveur `vsftpd` pour imposer des politiques de sécurité strictes (refus des connexions anonymes, forçage du SSL).

Perspective

En conclusion, ce projet illustre parfaitement que la sécurité informatique ne repose pas uniquement sur l'installation d'outils, mais sur une configuration rigoureuse et une vérification systématique. L'architecture mise en place constitue une base solide qui pourrait être étendue, par exemple, en remplaçant l'authen-

tification par mot de passe par une authentification par certificat client (Mutual TLS), renforçant encore davantage la sécurité du système.